

OneBlock - A Blockchain Student Portal

Project Team

Zain Ul Abideen	21I-1551
Aitsam Tariq	21I-1583
Sameer Saif	21I-0757

Session 2021-2025

Supervised by

Mr./ Ms./ Dr. Qaiser Shafi

Co-Supervised by

Mr./ Ms./ Dr. Supervisor Name



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Client Web Application	2
1.3.2	Blockchain Log Collector	3
1.3.3	Data Visualization and Reporting	3
1.3.4	Admin Web Application	3
1.4	User Classes and Characteristics	4
2	Project Requirements	7
2.1	Use-case/Event Response Table/Storyboarding	7
2.2	Functional Requirements	9
2.2.1	Client Web Application	9
2.2.2	Blockchain Log Collector	10
2.2.3	Data Visualization and Reporting	10
2.2.4	Admin Web Application	11
2.3	Non-Functional Requirements	11
2.3.1	Reliability	11
2.3.2	Usability	12
2.3.3	Performance	12
2.3.4	Security	12
2.4	Domain Model	13
3	System Overview	15
3.1	Architectural Design	15
3.2	Design Models	18
3.3	Data Design	24
3.3.0.1	Key Data Structures:	24
3.3.0.2	Data Storage:	25
	References	27

List of Figures

2.1	Use Case Diagram	8
2.2	Domain Model	13
3.1	Box and Line Figure	16
3.2	System Architecture	17
3.3	DFD	19
3.4	State Transition	20
3.5	Activity Diagram	22
3.6	Class Diagram	23

List of Tables

1.1	User Classes and Characteristics	5
2.1	Event Response Table	9

Chapter 1

Introduction

The OneBlock blockchain-based student portal is designed to securely manage student audit logs, addressing concerns about data security, integrity, and transparency in educational institutions. This document is intended for various stakeholders, including developers who will build and maintain the system, project managers overseeing its development, marketing staff promoting its features, users such as students and faculty interacting with the system, testers ensuring functionality and security, and documentation writers creating user guides. The system's use of blockchain technology ensures tamper-proof, decentralized data management, enhancing trust and accountability. [5].

1.1 Problem Statement

The trend toward cyber technology-based education has also necessitated that academic institutions effectively use centralized systems to store and manage large amounts of sensitive student data. It is a great concern to note that such systems are vulnerable to cyber attacks, illegal alteration of data, and system failures. These issues lead to a lack of faith in the system and even in the data stored. Right now several educational institutions grapple with issues such as log obfuscation, slowness in the response time to harmful actions, and the leaking of crucial information.

This blockchain-based online application seeks to implement solutions to combine these attributes and host a virtual decentralized application to store student account logs and records with foolproof protection against tampering using smart contracts, even erase platform restrictions, the system will make it impossible to change data once when it is recorded undetected. Also, the system will actively check the entries, identifying illegal changes in the records by warning various levels, and stop failures occurring at one point in the system. [1]

1.2 Scope

The principal aim of this project is to create a web application based on blockchain technology that enables educators to safely oversee and track student-related information and actions. The application's main goal is to offer a tamper-proof log system with immutable records saved on the blockchain with smart contracts to guarantee data integrity. The logs will gather data from various events about student activities, administration actions, and faculty inputs to prevent unauthorized additions or deletions. The data will be stored in a decentralized fashion. The solution reduces the danger of a single point of failure and improves availability by spreading the data among several nodes.

One important component that will allow for live tracking of system activity with automated anomaly identification is real-time monitoring. Unusual activity or unauthorized modifications will result in immediate notifications, guaranteeing prompt reactions to possible security risks. Furthermore, the program will facilitate faculty members in publishing announcements straight through the system, offering a more efficient means of communication for notifications and changes.

The system will include data visualization elements, such as graphs, charts, and log tables with filtering options, to further improve user experience and make it simpler for users to study data and spot trends. Every time something changes, thorough reports will be produced, with an emphasis on detailing abnormalities or other noteworthy occurrences. In addition, the project will guarantee compliance with data privacy and security laws and be compatible with the university's current infrastructure. The scope of this solution will finally address the major issues of data integrity, availability, and transparency by offering a transparent, safe, and effective platform for managing student data. [3].

1.3 Modules

Write down the modules of the proposed project. Each module should highlight features, using bulleted/numbered notation. When developing both a mobile app and a web app, group the modules according to the system types, such as Client Web App, Client Mobile App, Admin Web App, etc. [9]. [7]

1.3.1 Client Web Application

The main interface via which faculty members will communicate with the system will be this module. They will be able to keep an eye on student-related activities, go over logs, post notices, and get notifications instantly thanks to the web application. Teachers will be able to produce reports using the information gathered.

1. Secure login and authentication for faculty.
2. Dashboard for real-time monitoring and alerts.
3. Tamper-proof log viewing and filtering options.
4. Announcement feature for faculty updates.

1.3.2 Blockchain Log Collector

Logs from a variety of sources, such as student activities, administration decisions, and system modifications, will be gathered by this module. The blockchain will be used to securely store the gathered data, guaranteeing decentralized access and immutability.

1. Real-time log collection and storage on blockchain nodes.
2. Smart contracts for handling changes to logs.
3. Immutable data storage ensures no deletions or tampering.
4. Decentralized distribution of logs for enhanced availability.

1.3.3 Data Visualization and Reporting

The primary goal of this module is to provide professors and administrators with a quick overview of trends, anomalies, and system activities by using graphs, charts, and tables to illustrate the recorded data. Additionally, it will enable the creation of thorough reports using the data.

1. Graphs and charts for visualizing logs and trends.
2. Generation of detailed reports on alerts and system changes.

1.3.4 Admin Web Application

With the help of this module, administrators will be able to control more aspects of the system, including user roles, blockchain node management, and log viewing.

1. User management, including roles and permissions.
2. Node and network management for blockchain.

[2]

1.4 User Classes and Characteristics

Identify the various user classes that you anticipate will use this product, and describe their pertinent characteristics. [6]

Table 1.1: User Classes and Characteristics

User Class	Description
Student	The primary users of the system, students can view their academic logs, such as submissions, attendance, and grades. Students will also receive announcements from faculty and can check the status of their activities within the blockchain. Most students have limited technical knowledge and interact with the system via a web-based interface.
Faculty	Faculty members are responsible for inputting and managing student data into the blockchain. They can post announcements, generate reports, and monitor logs for anomalies. Faculty will need a moderate technical understanding of the system and will primarily use a secure web portal to interact with the logs and announcements.
Administrators (Admin)	System administrators are responsible for managing user accounts, monitoring blockchain nodes, and ensuring system security. They have advanced technical knowledge and access to high-level features such as smart contract configurations and user management. Admins ensure the system operates smoothly and securely, making adjustments as needed.
Blockchain Network Participants (Validators)	Validators are entities within the blockchain network responsible for verifying and validating transactions (logs) in the system. They ensure that all data entered into the blockchain is secure, tamper-proof, and follows the smart contract rules. Validators operate at the infrastructure level and have no direct interaction with the user interface.
Testers	Testers are responsible for evaluating the functionality, security, and performance of the system. They conduct both manual and automated tests to identify bugs, and security vulnerabilities, and ensure that the system meets its non-functional requirements. Testers require a good understanding of blockchain and the system's core functionalities.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

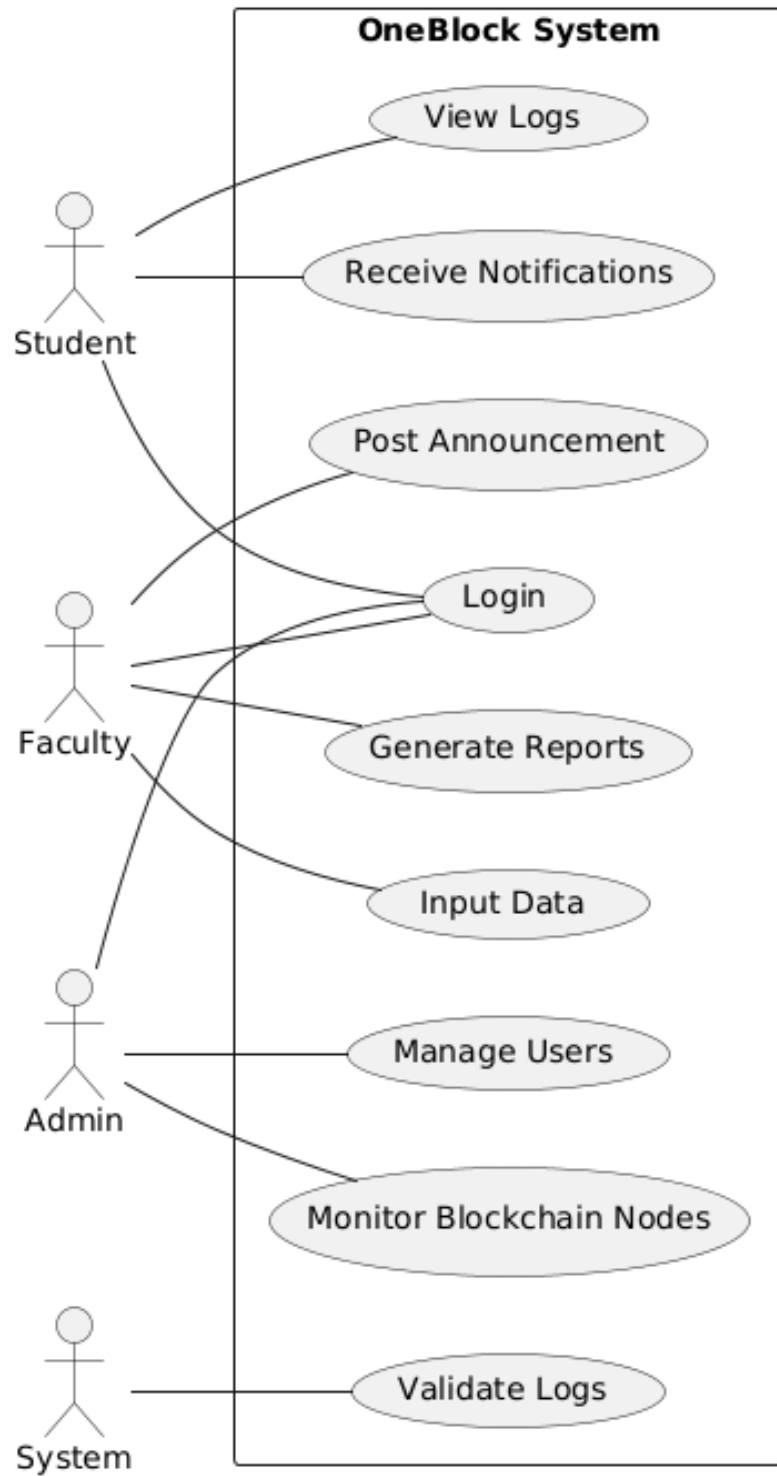


Figure 2.1: Use Case Diagram

Table 2.1: Event Response Table

Event	Trigger	System Response
Student views data	Student logs into the system	The system retrieves student-specific data from the database and displays it on the dashboard.
Faculty inputs student data	Faculty submits data (grades, attendance)	The system creates a new log entry and triggers a smart contract to store the data immutably in the blockchain.
Admin monitors node status	Admin checks node status	The system displays the health status of blockchain nodes, highlighting any failed nodes for corrective actions.
Anomaly detected in logs	Blockchain detects suspicious log activity	The system alerts admins, automatically logs the anomaly, and distributes the notification to relevant users.
Announcement posted	Faculty posts an announcement	The system distributes the announcement via email and in-app notifications to all relevant students.
Log validation request	New log entry submitted	The system runs the smart contract validation to ensure the log's integrity and immutability, and it stores the log across blockchain nodes.

2.2 Functional Requirements

This section describes the system's functional requirements in the natural language style. This section is typically organized by feature as a system feature name and specific functional requirements associated with this feature. It is just one possible way to arrange them. Other organizational options include arranging functional requirements by use case, process flow, mode of operation, user class, stimulus, and response depending on the technique used to understand functional requirements. Hierarchical combinations of these elements, such as use cases within user classes, are also possible. [4]

2.2.1 Client Web Application

The following are the requirements for the Client Web Application module:

1. The system must provide secure login and authentication for faculty members to access the web portal.

2. Faculty must be able to post announcements directly through the web application for student notifications.
3. The system must allow faculty to generate detailed reports on student activities, logs, and anomalies.
4. Faculty must be able to post announcements directly through the web application for student notifications.
5. Admin should have filtering options for logs based on hashes, instructor name, or browser used.

2.2.2 Blockchain Log Collector

The following are the requirements for the Blockchain Log Collector module:

1. The system must collect and store logs from student activities, faculty actions, and administrative decisions in real time using blockchain technology.
2. The system must ensure all stored logs are immutable, meaning they cannot be altered or deleted once recorded on the blockchain.
3. The system must distribute logs across multiple blockchain nodes to ensure data availability and prevent a single point of failure.
4. The system must validate logs using smart contracts to detect and flag suspicious or unauthorized changes automatically.
5. The system must provide decentralized access to logs, ensuring no single entity can tamper with the data without consensus from the blockchain network.

[8]

2.2.3 Data Visualization and Reporting

The following are the requirements for the Data Visualization and Reporting module:

1. The system must provide graphical representations of log data using charts, graphs, and tables for easier analysis of trends and anomalies.
2. The system must enable real-time updates of visual data, reflecting any changes or new logs immediately in the graphs and charts.

3. The system must allow exporting reports and visual data in standard formats like PDF, CSV, and Excel for offline review.
4. The system must provide a detailed breakdown of anomalies or suspicious activities detected by the blockchain in a visual format.

2.2.4 Admin Web Application

The following are the requirements for the Admin Web Application module:

1. The system must provide administrators with secure login and role-based access control to manage user accounts and permissions.
2. Admins must have access to an audit trail that details all system changes, user permissions, and administrative actions.
3. The system must log all notifications and alerts sent, allowing administrators to review the history of alerts and user responses.

2.3 Non-Functional Requirements

This section specifies nonfunctional requirements. These quality requirements should be specific, quantitative, and verifiable. The following are some examples of documenting guidelines.

2.3.1 Reliability

1. The system must achieve a Mean Time Between Failures (MTBF) of at least 500 hours, ensuring minimal disruptions.
2. The system must be able to recover from any failure within 15 minutes to restore normal operations.
3. The system should provide automated error detection and correction mechanisms to identify and resolve faults before they impact users.
4. All critical system components (e.g., blockchain nodes, smart contracts) must include failover strategies to minimize the impact of failures on availability.

2.3.2 Usability

1. The system must allow users to perform key tasks (e.g., log viewing, report generation) with no more than three interactions.
2. Faculty and students should be able to learn the core functions of the system in less than 30 minutes of use.
3. The interface must provide clear error messages and suggestions to recover from errors, with an error recovery rate of 90 percent within a single attempt.
4. System responsiveness (e.g., loading logs or dashboards) should not exceed 2 seconds for any action.

2.3.3 Performance

1. The system must handle up to 1,000 concurrent users without degradation of performance.
2. All critical transactions (e.g., blockchain log entry or update) must be processed within 5 seconds.
3. Data retrieval actions (e.g., log queries, report generation) must have a response time of less than 3 seconds for 95 percent of queries.
4. The system should support data visualization of up to 10000 log entries without performance issues.

2.3.4 Security

1. The system must use blockchain technology to guarantee 100 percent immutability of student and system logs.
2. Unauthorized access attempts should be logged, with an alert sent to the admin within 5 seconds of detection.
3. The system must be resilient against DDoS attacks, maintaining service availability at 99 percent uptime during such attacks.
4. All sensitive data must be encrypted to ensure the privacy and security of student information.

2.4 Domain Model

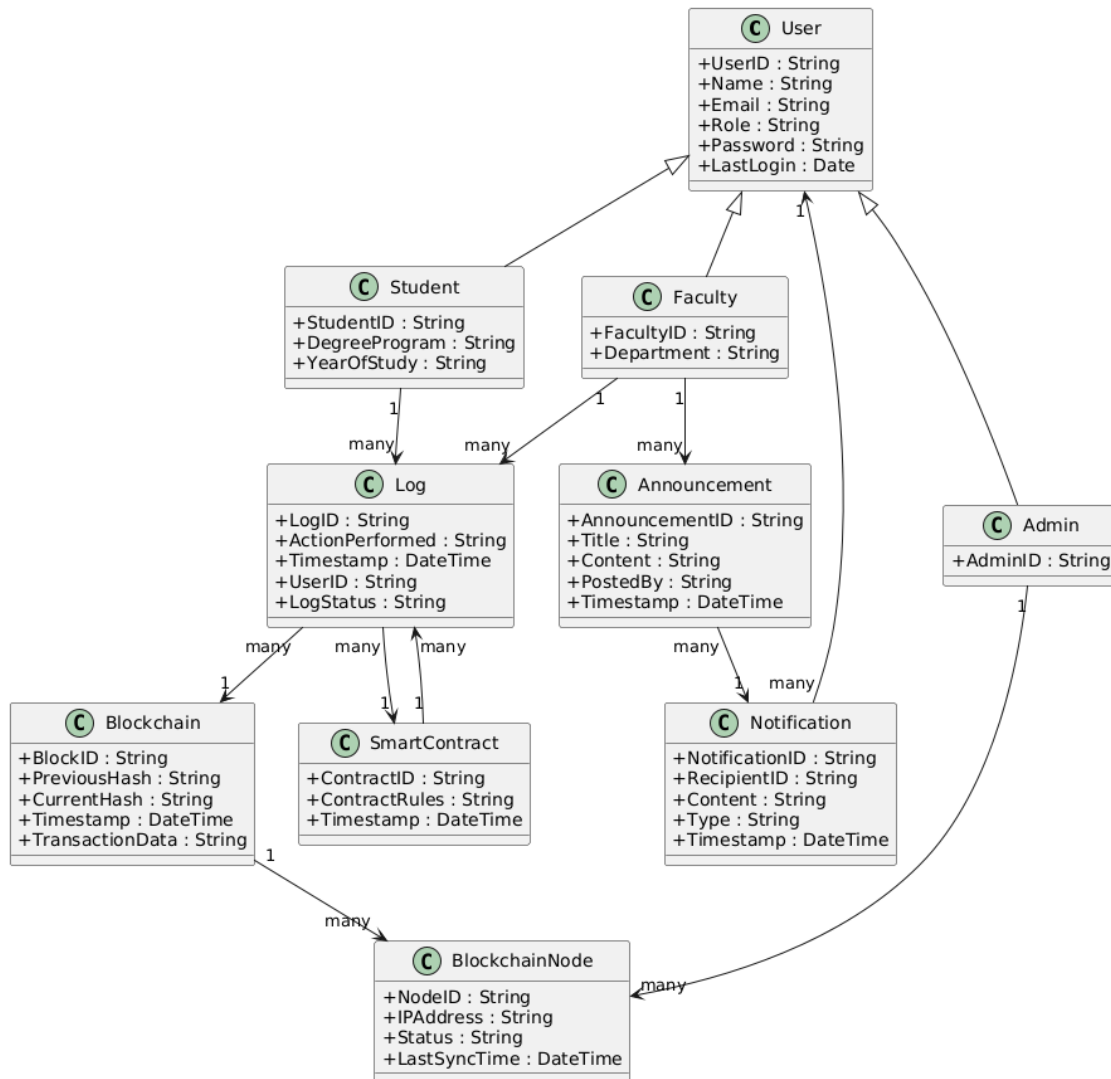


Figure 2.2: Domain Model

Chapter 3

System Overview

This chapter refers to the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It's a broad field that encompasses many aspects of creating a complex system, including hardware, software, network, and data. The goal is to design a system that is efficient, reliable, and scalable while meeting user needs and organizational goals. The process often involves understanding and defining system requirements, creating architectural diagrams, selecting appropriate technologies, and planning for integration with other systems.

3.1 Architectural Design

The OneBlock architecture is designed to ensure tamper-proof record-keeping within the student portal. Instructors interact with the portal's front end to manage courses and input student data (grades, attendance). This data is then forwarded to the backend which triggers the Log Generation process, which extracts and formats relevant log entries. These formatted logs are then securely transmitted to the Ethereum Blockchain via the Ethereum node, where the OneBlock Smart Contract stores them immutably. Administrators access the OneBlock Website to view, analyze, and verify these blockchain-secured logs, ensuring transparency and accountability within the educational environment.

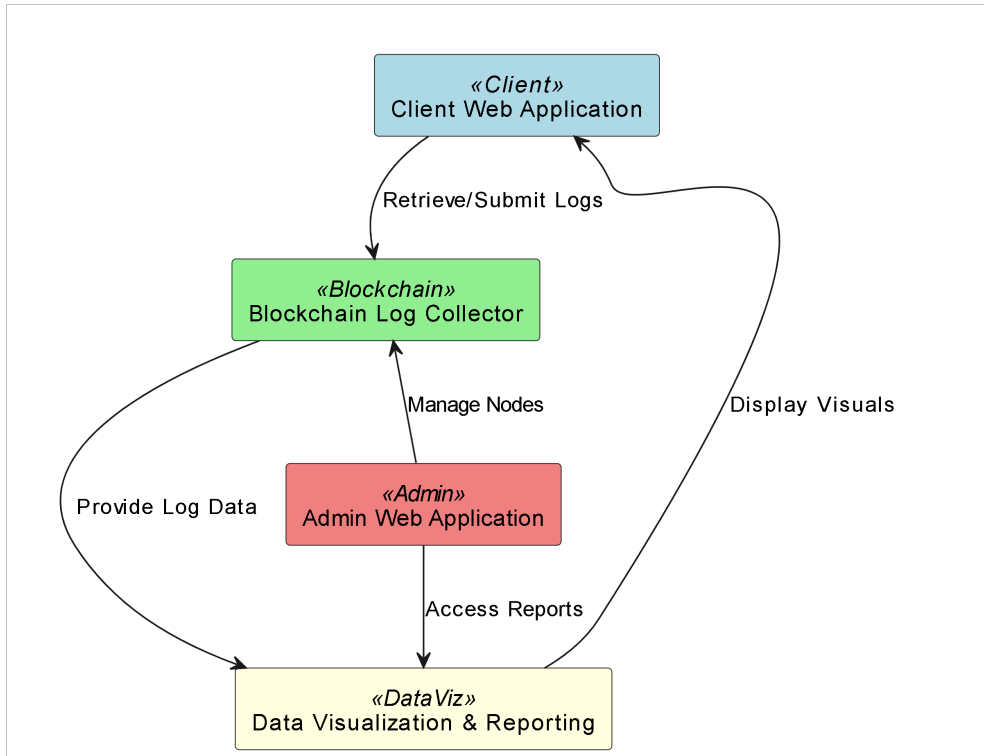


Figure 3.1: Box and Line Figure

The core modules include:

1. **Client Web Application** This interface is used by faculty to log in, monitor student activities, generate reports, and post announcements. It enables real-time monitoring of logs and alerts.
2. **Blockchain Log Collector** The system's core is responsible for collecting and securely storing logs of student activities, administrative actions, and system changes on a decentralized blockchain, ensuring immutability. Smart contracts validate logs and detect suspicious activities.
3. **Data Visualization & Reporting** This module generates visual reports (charts, graphs, tables) from blockchain logs, allowing faculty and administrators to quickly analyze trends, anomalies, and system performance.
4. **Admin Web Application** This module enables system administrators to manage users, monitor blockchain nodes, ensure system integrity, and access detailed reports, supporting security and operational configurations.

System Architecture of OneBlock Blockchain-Based Student Portal The architecture of OneBlock is divided into four distinct layers:

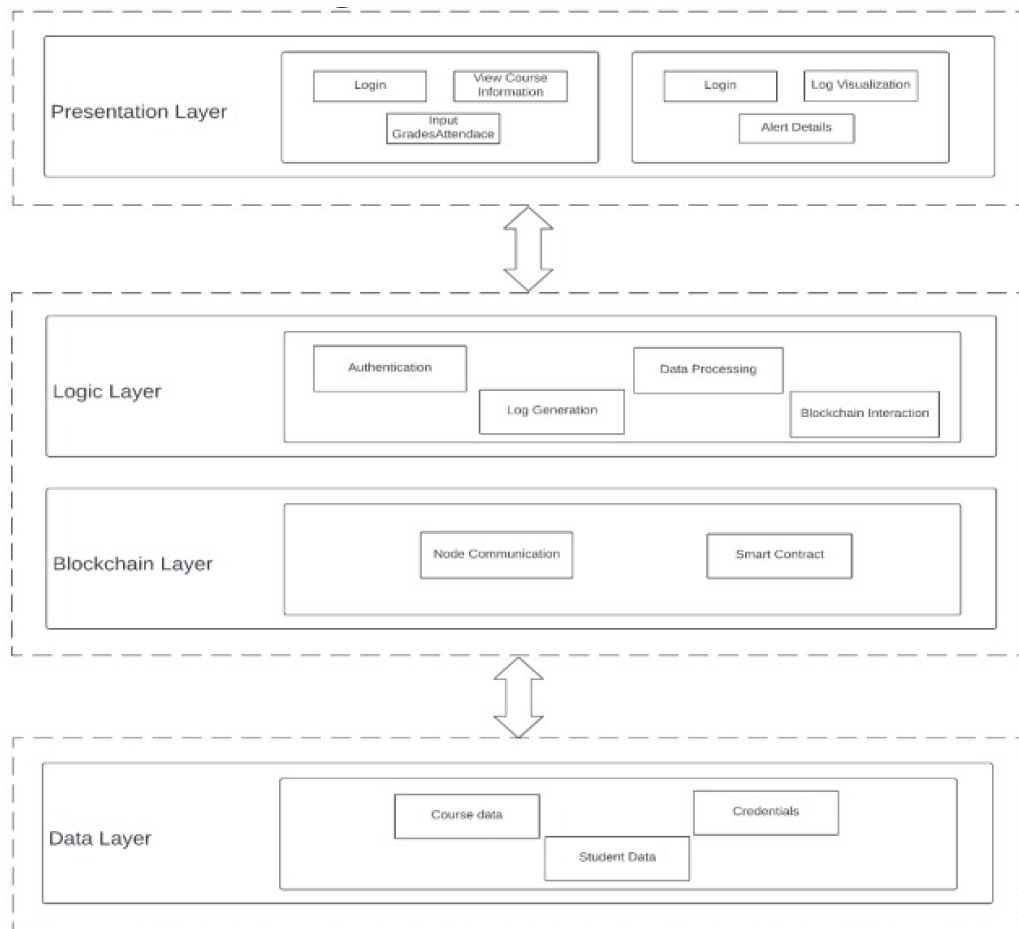


Figure 3.2: System Architecture

Presentation Layer: This is the user-facing layer that provides interaction points for both students and faculty. Features include secure login, course information, and log visualization for system alerts and activity. Users can input grades, and attendance, or view details, while faculty can also manage announcements.

Logic Layer: This layer handles the core functionalities of the system, including authentication, log generation, data processing, and interaction with the blockchain. It ensures that data from user actions and system processes are appropriately handled and transferred securely to the blockchain.

Blockchain Layer: The backbone of the system, responsible for decentralizing and securing the data. It manages node communication and smart contracts, ensuring that logs are immutable and securely distributed across the blockchain.

Data Layer: This layer stores the essential data like course information, student records, and credentials. The data is utilized by both the logic and blockchain layers to ensure that only authorized actions are recorded and maintained.

3.2 Design Models

DFD

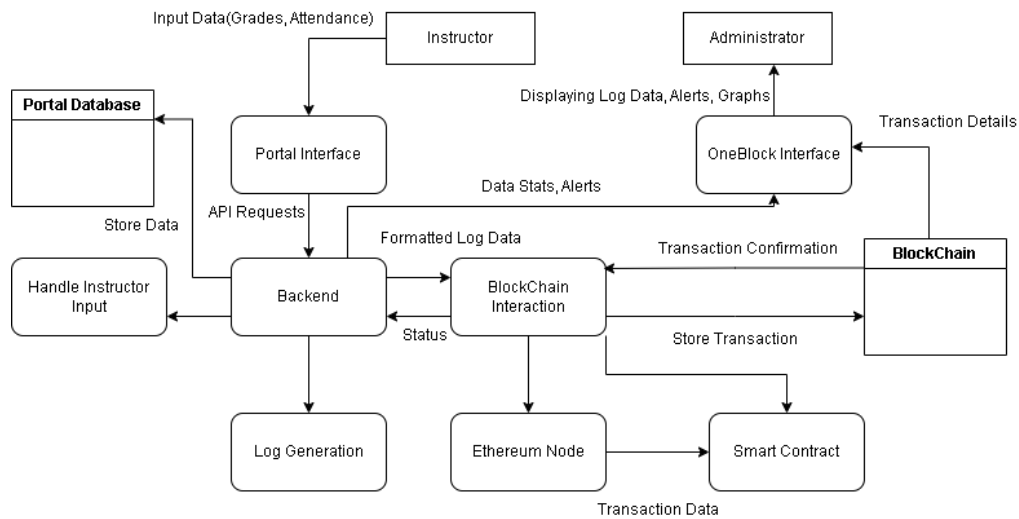


Figure 3.3: DFD

The data flow diagram illustrates the movement of information within the OneBlock system. Instructors interact with the custom LMS, inputting student grades and attendance data, which is stored in the LMS database. The backend then extracts and formats this information, sending it as transactions to the Ethereum Blockchain. These transactions are processed by the OneBlock smart contract, permanently recording the log data on the Blockchain. Finally, the OneBlock website retrieves this blockchain-stored data, providing administrators with a clear view of the logged information, accompanied by relevant statistics and alerts.

State Transition Diagram

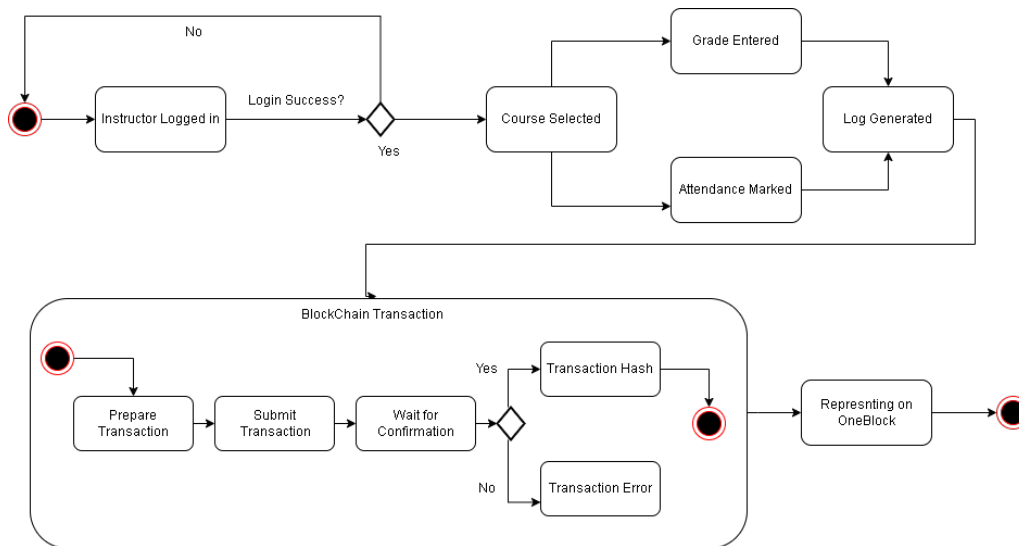


Figure 3.4: State Transition

The state transition diagram illustrates the different states that are completed within the OneBlock system, along with the events that trigger transitions between these states. The instructor begins in the "Logged Out" state and can enter the LMS upon successful login. Once logged in, they can select a course, after which they choose to either mark attendance or enter grades. If the specific criteria are met, these actions result in logs being generated and successfully stored on the blockchain. This log entry is then prepared as a blockchain transaction, signed for security, and sent to the Ethereum node using Web3.js. The Ethereum node broadcasts the transaction to the network, where it is validated and added to a block on the blockchain. On the OneBlock website get the information about the transaction.

Activity Diagram

The activity diagram demonstrates the process flow for an instructor logging into the system and performing key tasks. It starts with the instructor logging in, after which they select a course. The system then provides two options: either mark attendance or mark assessments. Once the instructor completes either task, the system generates a log. Finally, the log is sent to the admin portal for review, marking the end of the process. This diagram ensures transparency and accountability for faculty actions within the OneBlock system.

Activity Diagram

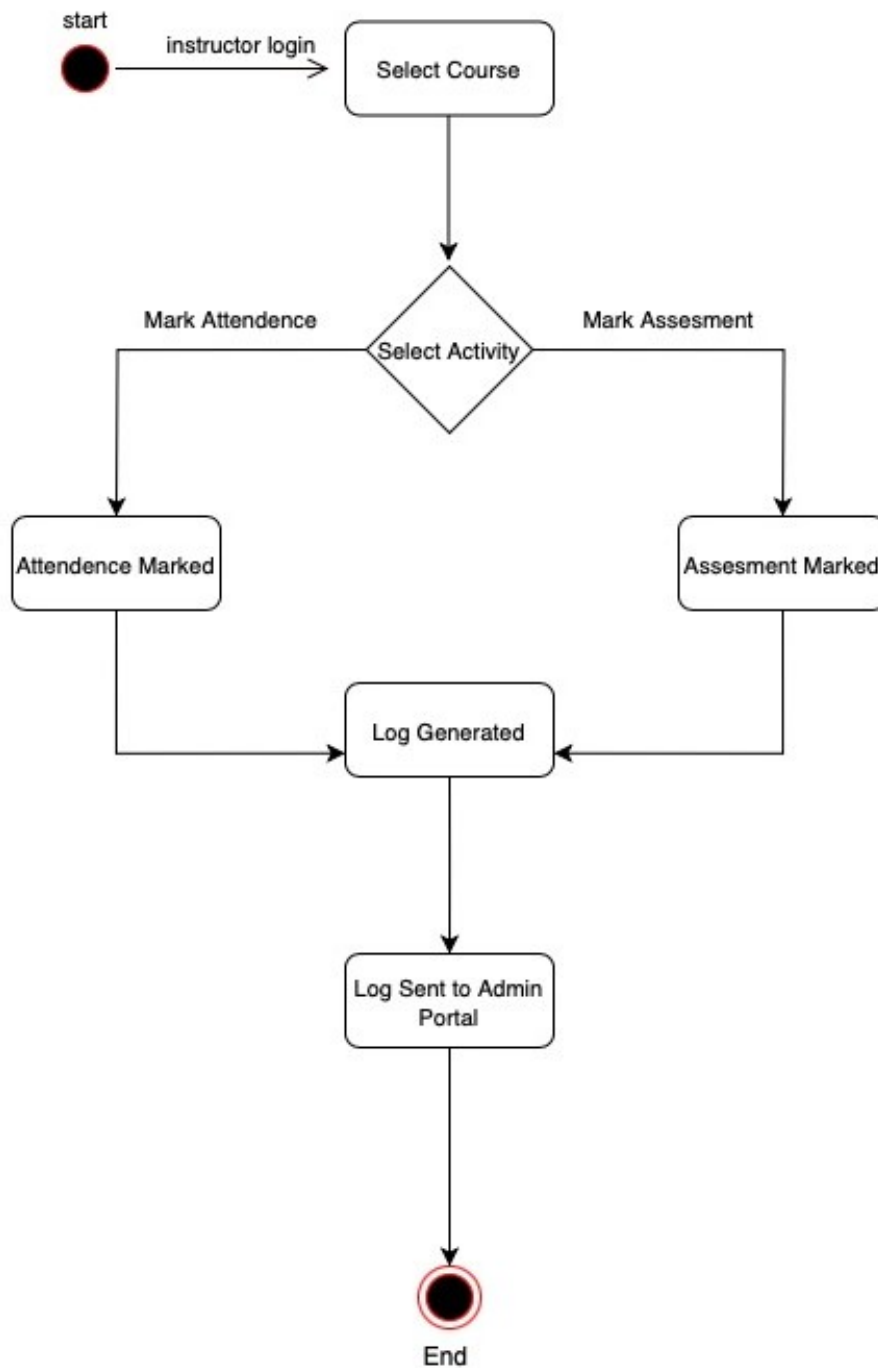


Figure 3.5: Activity Diagram

Class Diagram

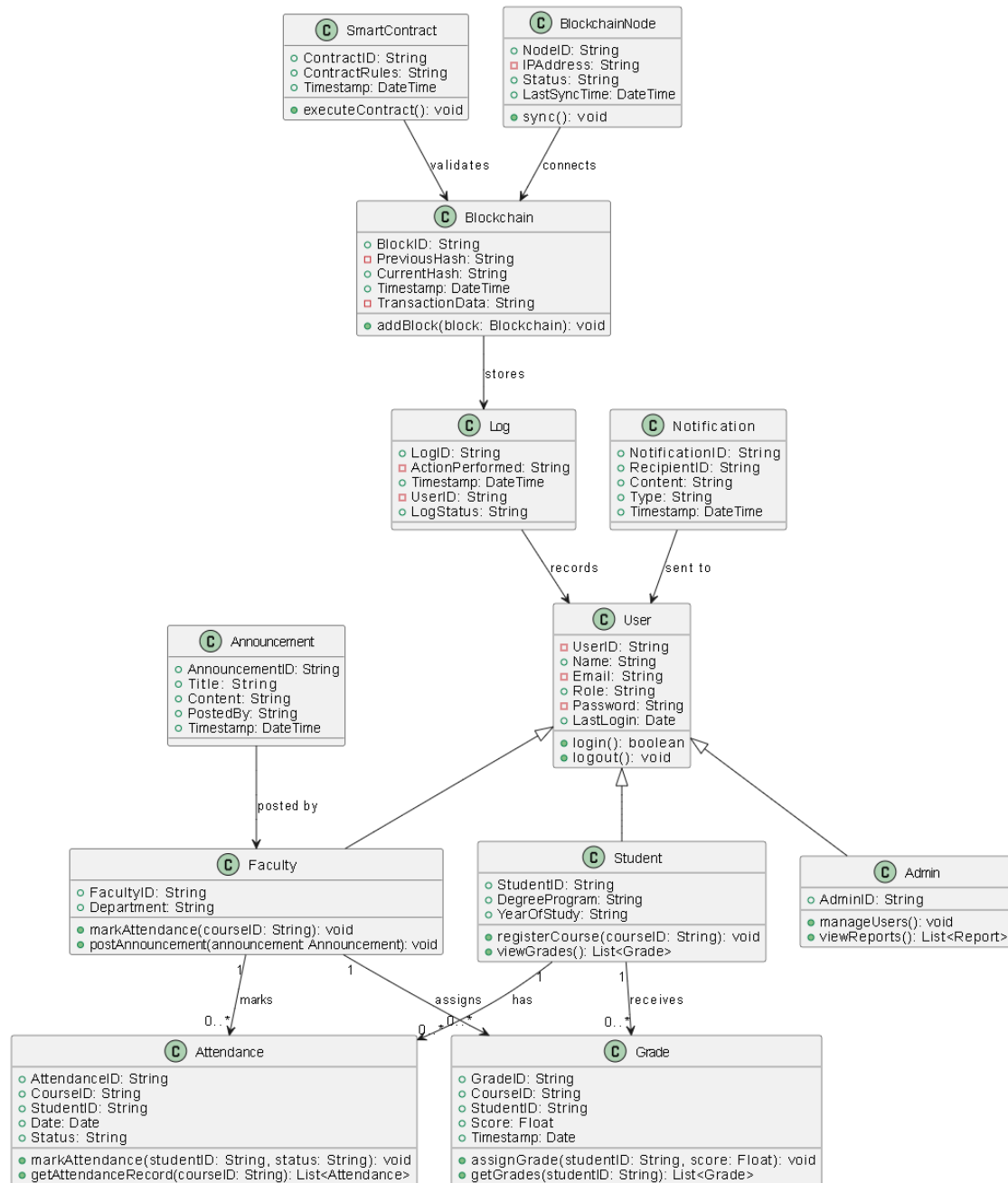


Figure 3.6: Class Diagram

The class diagram for the OneBlock system illustrates the system's structure by defining key classes, their attributes, methods, and relationships. It captures the interactions between different entities involved in the application.

1. **User Hierarchy:** The diagram features a base **User** class, with three specialized subclasses: **Student**, **Faculty**, and **Admin**. Each subclass inherits common prop-

erties from the User class while also defining unique attributes and methods relevant to their specific roles.

2. **Attendance and Grades:** The Attendance class captures student attendance records, with a **1 to many** relationships to the Student class, indicating that each student can have multiple attendance entries. Similarly, the Grade class allows tracking of student grades, establishing a **1 to many** relationship with both Student and Faculty classes, reflecting the ability for each faculty member to assign grades to multiple students.
3. **Logging and Notifications:** The Log class records user activities, associated with the User class through a **1 to many** relationships, showing that multiple logs can be generated by a single user. The Announcement class is linked to the Faculty class, denoting that faculty members can post announcements.
4. **Blockchain and Smart Contracts:** The diagram also includes components related to the blockchain infrastructure, such as the Blockchain, SmartContract, and BlockchainNode classes, which facilitate secure data storage and validation within the system.
5. **Relationships and Cardinality:** The relationships are defined with cardinality annotations, clearly indicating how instances of each class interact with one another (e.g., one-to-many and many-to-one relationships).

3.3 Data Design

The **OneBlock** system leverages blockchain technology for secure log storage and relational databases for managing user data, course information, and other structured content. The data design ensures that all information is stored securely, processed efficiently, and organized for real-time access and analysis.

3.3.0.1 Key Data Structures:

1. Blockchain-Based Log Storage:

- **Log Blocks:** The blockchain stores logs of student activities, faculty actions, and system changes in an immutable structure. Each log entry is stored in a block, which includes:
 - Timestamp of the event
 - Activity type (e.g., login, assessment, attendance)
 - User ID (instructor or student)

- Course or assessment details
- Hash of the previous block for secure chaining
- **Smart Contracts:** Smart contracts validate the integrity of logs, ensuring that logs are automatically recorded upon specific events (e.g., marking attendance or grading assessments). These smart contracts ensure tamper-proofing and data accuracy by preventing manual log manipulation.

2. Relational Databases:

- **User Information Database:**
 - Stores user data, including faculty, students, and administrators.
 - Fields include user ID, name, role (e.g., faculty or student), credentials (securely hashed passwords), and permissions.
- **Course Database:** Stores course-related data such as course ID, course name, instructor ID, student list, and relevant time schedules.
- **Attendance and Assessment Records:**
 - A separate table for attendance logs where the instructor's actions (marking attendance) are recorded concerning the blockchain log entry.
 - Assessment records table storing grades, submissions, and evaluations, also linked to blockchain for traceability.

3. Data Visualization & Reporting Module:

- **Analytical Data Structures:** In-memory data structures for aggregating, analyzing, and visualizing logs. These structures pull data from the blockchain and databases to generate charts and reports.
- **Graphical Reports:** The system extracts specific fields (e.g., timestamps, activity types) from the blockchain and presents them in visual form to administrators and faculty.

3.3.0.2 Data Storage:

- **Blockchain (Decentralized Ledger):** Primarily used to store logs, ensuring they are immutable and tamper-proof.
- **Relational Databases (FireBase):** Used to store structured data like instructor information, course details, student names, and assessments.

Bibliography

- [1] Alexander Grech and Anthony F. Camilleri. Blockchain for education: Opportunities, challenges, and future directions. *JRC Science for Policy Report*, 2017.
- [2] J. E. Holt. Logcrypt: Forward security and public verification for secure audit logs. *Journal Title*, 2006.
- [3] M. Kumar, A. K. Singh, and S. Kumar. Secure log storage using blockchain and cloud infrastructure. *Journal Title*, 2021.
- [4] Antonio Panarello, Nadra Tapas, Giovanni Merlino, Francesco Longo, and Antonio Puliafito. Blockchain-based security solutions for the internet of things: A survey. *IEEE Internet of Things Journal*, 5(6):4857–4876, 2018.
- [5] T. Pulls and R. Peeters. Balloon: A forward-secure append-only persistent authenticated data structure. In *Computer Security – ESORICS 2015*, 2015.
- [6] Mike Sharples and John Domingue. Blockchain for decentralized education system: A new approach. In *Proceedings of the 11th International Conference on Learning Analytics and Knowledge (LAK '16)*, pages 409–416. ACM, 2016.
- [7] L. Shekhtman and E. Waisbard. Engravechain: A blockchain-based tamper-proof distributed log system. *Future Internet*, 2021.
- [8] John Singh and Shubhangi Singh. Blockchain-based smart contracts: A systematic mapping study. *Electronic Commerce Research and Applications*, 36:100858, 2019.
- [9] F. Von Eye, D. Schmitz, and W. Hommel. A secure logging framework with focus on compliance. Available: www.iaria.org.